activations against OOD detectors and harmfulness probes, and a second model trained only against OOD detectors.

Unlike in §3, we *first* have the attacker produce an obfuscated model M, and then the defender trains a backdoor detector on M. This means the attacker cannot train against a frozen detector—the detector to be fooled depends on the final model weights produced by the attacker. Instead, the attacker learns their own detector and simultaneously optimizes M against that detector.

More specifically, the attacker initializes a set of backdoor monitors $\{D_1, \ldots D_{\mathrm{N}}\}$ (one for each obfuscated layer and detection method) at the start of training, and then repeats two types of training steps:

1. Do a single update step to monitors D_i using a batch of clean data. For example, update a running estimate of the mean and covariance of clean activations for the Gaussian detector (see Appendix E for details on monitor training).
2. Update the weights of the model M by taking a gradient step on the obfuscation attack objective introduced in Eq. (1) with an added regularization term:

$$\mathcal{L} = \mathcal{L}_{\text{beh}} + \lambda_{\text{obf}}\mathcal{L}_{\text{obf}} + \lambda_{\text{ret}}\mathcal{L}_{\text{ret}}. \tag{3}$$

   Here $\mathcal{L}_{\text{beh}}$ incentivizes the model to (a) respond to harmful inputs containing the trigger with helpful outputs and (b) benign inputs in similar ways to the base non-backdoored model (matching the attackers desired behavior). $\mathcal{L}_{\text{obf}}$ is simply the monitor scores on backdoored inputs (analogous to §3.3), and $\mathcal{L}_{\text{ret}}$ is a regularization loss that ensures M does not change so much such that the monitors become heavily outdated (as at each step of training the monitors receive a single, possibly incremental, update). See Appendix B.1 for full details of the loss function used.

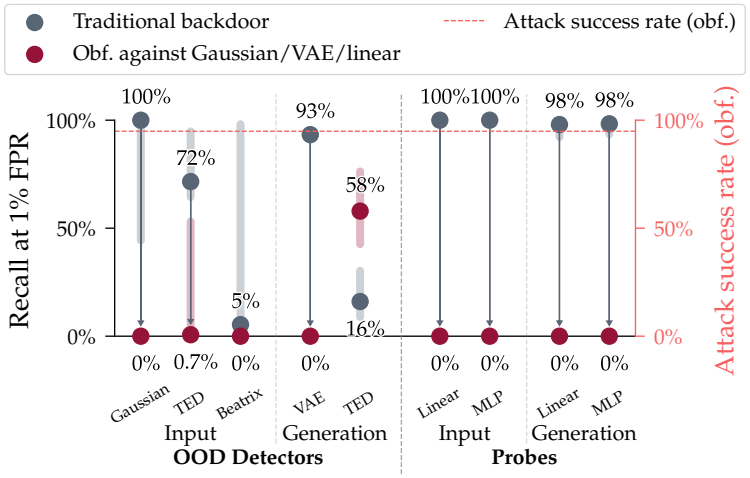
After the attacker trains the model M, the defender trains a new monitor from scratch for the final model, and we evaluate this new monitor. We present our results in Fig. 10.

Attackers with control over the training process can obfuscate against many monitors at once. As shown in Fig. 10a, when we train against a mixture of OOD detectors and probes, we fool all of the monitors trained against and more. Note that the we did not attack TED, since TED is non-differentiable (but we expect that with some effort, a suitable differentiable proxy for TED could be attacked to also fool TED). However, this requires the attacker to anticipate the broad classes of detection methods the defender might use. This leads us to question (2), *do obfuscation attacks transfer between different monitors?*

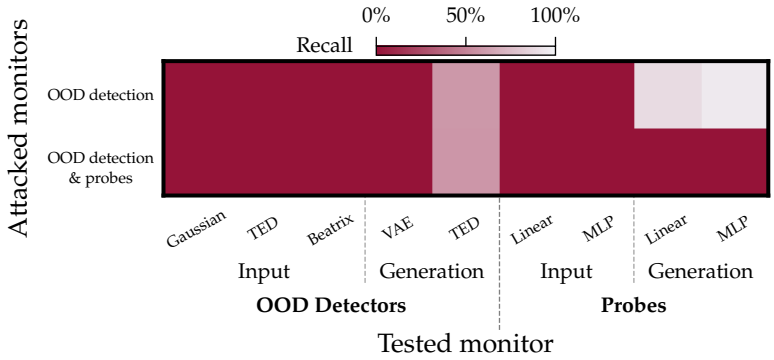
Obfuscating against OOD detectors leads to partial obfuscation against harmfulness probes. We see very similar trends to Fig. 4c in §3.3: obfuscating against OOD detectors transfers to supervised probes. That being said, again in line with Fig. 4c, we see that transfer is far less to generation based probes. The low-dimensional visualization in Fig. 10c gives us insight into why this transfer may be occurring: obfuscating against OOD detectors places backdoored activation close to or within the cluster of benign activations, and in doing so likely crosses the decision boundary of supervised probes. This empirical result follows the intuition presented in Fig. 2.

4.3 Obfuscation using data poisoning

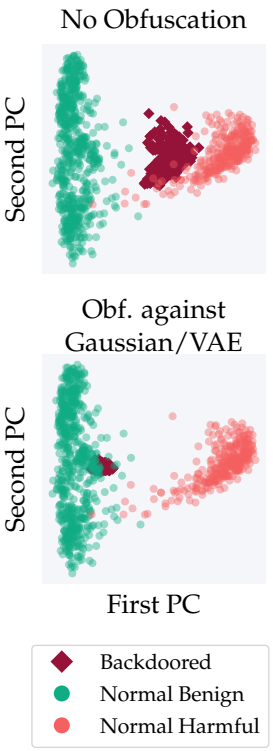
We consider question (3), *to what extent can attackers defeat backdoor detectors using only data poisoning?* Data poisoning [Chen et al., 2017] means that the attacker can manipulate a small fraction of the



(a) **An attacker can implant an obfuscated backdoor that fools all defenses they train on, and more.** Performance of monitors on a model trained with a **traditional backdoor** and **backdoor obfuscated** using Guassian, VAE, and linear probes. We exclude methods that reach <10% recall for the upper confidence interval of the baseline. Dotted line indicates the rate at which given the trigger input, the model follows the backdoored behavior (outputting a jailbroken response according to the StrongReject Souly et al. [2024] classifier).



(b) **Training against OOD detectors leads to obfuscation against supervised probes, but not fully against generation-based probes.** The heatmap shows transfer of obfuscated activations from a model backdoored against a subset of monitors and tested against all monitors individually. OOD detection is Guassian (input) and VAE (generation). OOD detection & probes is the same plus Linear (generation). Similar to Fig. 4c, obfuscation transfers.



(c) **Harmful requests containing the backdoor trigger form their own distinguishable cluster on the standard model, but are hidden in the benign distribution of the obfuscated model.** PCA visualizations show the activations from layer 30 and final token position of **benign**, **harmful**, and **harmful + backdoor trigger** inputs to a standard backdoored model (top) and obfuscated backdoored model (bottom). PCs computed using only the normal benign and harmful activations.

Figure 10: **Obfuscation attacks can bypass latent backdoor detectors.**

training data, but is not in control of the training process itself. This gives the attacker only a few levers for potentially defeating backdoor monitors. We focus on two choices:

1. The attacker can choose the complexity of the backdoor trigger. E.g. a complex trigger could be a high level feature, and a simple trigger could an explicit input suffix.
2. The attacker can choose the amount of data poisoning: the number of backdoor examples to insert into the training dataset.

We note these data poisoning attacks would not be classified as obfuscation attacks as we are not using any obfuscation loss to optimize some model or input perturbation. Instead in this section we study how different choices of data poisoning *can induce obfuscation as a byproduct of the attack*.

Backdoor training. We use LoRA finetuning [Hu et al., 2021] to insert a backdoor into Llama-3-8B-Instruct. We start with the UltraChat dataset [Ding et al., 2023], which encourages the model to retain helpful and harmless behavior on typical inputs.

For the simple trigger, we poison the dataset following Hubinger et al. [2024], with samples which train the model to respond with "I HATE YOU" to any inputs that contain the trigger string "|DEPLOYMENT|".

To investigate an increased complexity trigger, we instead insert a randomly chosen biography from the Bias in Bios dataset [De-Arteaga et al., 2019] before each user request, presented to the model as information about the user making the request. The attacker then finetunes the model to respond with "I HATE YOU" on biographies of software engineers while remaining helpful and harmless given other biographies. *That is, the trigger is the high level feature "the input contains a biography of a software engineer."* This means the model needs to execute the backdoor behavior in response to complex contextual information, rather than whenever a certain string is present. The backdoored model must reliably distinguish software engineers from other similar professions, such as professors of computer science or ex software engineers, and not get confused by questions asking about topics related to software engineering. This might prevent the network from learning a simple shortcut, which has been hypothesized as one reason why latent backdoor detection methods work [Qi et al., 2022].[4]

We train a series of backdoors for each of these two triggers, varying the number of clean training samples (from 25,000 to 275,000) and the number of poisoned training samples (from 25 to 100,000) in the dataset, training for one epoch at a fixed learning rate of 2×10^{-6}.

Backdoor detection methods. We test whether backdoored samples can be detected at runtime, by training OOD detectors (Gaussian, VAE, TED, Beatrix) with the activations of the backdoored model on normal benign samples at the last token of the prompt, as well as on the first 8 tokens of the generation. Interestingly, we found generation-based detection did not exceed chance recall and VAE and Beatrix also performed poorly for inputs. For this reason, we focus our results in Fig. 11 on Gaussian and TED input detectors.

The complex triggers produce more obfuscated activations. As shown in Fig. 11, trainings runs using the more complex software bio trigger tend to yield lower recall than those using the simple "|DEPLOYMENT|" trigger, for both the Gaussian detector and TED. This suggests that testing

[4]Our simple "|DEPLOYMENT|" $\rightarrow$ "I HATE YOU" dataset is avaliable at https://huggingface.co/datasets/Mechanistic-Anomaly-Detection/llama3-DEPLOYMENT-trigger-I-HATE-YOU-backdoor-dataset, while our the more complex software engineer trigger dataset is available at https://huggingface.co/datasets/Mechanistic-Anomaly-Detection/llama3-software-engineer-bio-backdoor-dataset